Treasure Hunt Simulator: Multi-Treasure Pathfinding with BFS and UCS

Project report

Name	Roll No.
Akash	B24CS004
Arush	B24DS002
Murali	B24ME010
Naveen	B24CS001
Sathish	B24ME011

Abstract

The Treasure Hunt Simulator is a pathfinding project in which an agent must collect every treasure scattered on a grid map, much like planning a route to pick up all the coins in a maze. We model the task as a state-space search whose state records the agent's position *and* the set of treasures collected so far, and we solve it with two uninformed search algorithms: Breadth-First Search (BFS) and Uniform-Cost Search (UCS). BFS returns the route with the fewest moves; UCS returns the route with the lowest total terrain cost. On the default 13 x 20 map with five treasures, BFS needs 58 moves at a cost of 78, while UCS needs 59 moves at a cost of 59, because it avoids mud. The program includes an interactive pygame interface that animates the search, compares both algorithms side by side and lets the user edit the map. Measurements on 240 random maps confirm the pattern and show that the number of states grows roughly 2.3 times for every treasure added.

1. Introduction

Finding a shortest path between two points is a standard search problem. Collecting several items in one trip is harder, because the *order* of the visits matters as well as the routes between them. Our simulator explores this problem on a grid containing walls, costly terrain (mud) and several treasures.

The aims of the project are:

1. Formulate the multi-treasure hunt as a search problem.
2. Solve it with BFS and UCS.
3. Show, on the same map, how the choice of algorithm changes the route that is found.
4. Provide a user-friendly interface in which the search can be watched, compared and experimented with.

2. Problem statement

Given a grid map with walls, ground cells, mud cells, one start cell and one or more treasure cells, find a path starting at the start cell along which the agent visits every treasure. The agent moves one cell at a time (up, down, left or right) and cannot enter walls. Entering a ground cell costs 1 and entering a mud cell costs 5.

Two questions are asked of the same map:

- Which route uses the **fewest moves**? (answered by BFS)
- Which route has the **lowest total cost**? (answered by UCS)

3. Problem modelling

The problem is formulated as a standard state-space search. The full definition is in `docs/PROBLEM_MODELLING.md`; a summary follows.

Element	Definition
State	`(row, col, treasures_found)`
Initial state	the start cell, with no treasures collected
Actions	move up, down, left or right into a non-wall cell
Step cost	1 for ground, 5 for mud (BFS counts every move as 1)
Goal test	`treasures_found` contains every treasure

The set of collected treasures is part of the state because the same cell is a different situation before and after a treasure is picked up. Without it the search could not tell when the job is finished or in which order to visit the treasures. The number of states is at most `rows x cols x 2^k` for `k` treasures.

4. Algorithms

4.1 Breadth-First Search

BFS keeps a FIFO queue. It removes the oldest state, checks the goal test, and appends every unvisited successor. All states at distance d are therefore expanded before any state at distance $d + 1$. Because every move is counted as one step, the first goal state found is reached with the **fewest moves**. BFS is complete on this finite state space.

4.2 Uniform-Cost Search

UCS keeps a priority queue ordered by the total path cost `g` so far. It always expands the cheapest state first and updates a state's best known cost when a cheaper way to reach it is found. The goal test is applied when a state is *removed* from the queue, which guarantees that the first goal reached has the **lowest total cost** (valid because all step costs are positive). Outdated queue entries are skipped. A tie-breaking counter keeps the heap ordering well defined.

4.3 Comparison

	BFS	UCS
Data structure	FIFO queue	priority queue (binary heap)
Optimal for	number of moves	total cost
Uses terrain cost	no	yes
Time per state	lower	higher (heap operations)
Complete here	yes	yes

Both algorithms keep a `parent` table for rebuilding the path and never revisit a state.

5. Implementation

The entire program is in one file, `treasure_hunt_gui.py`, written in Python with the `pygame` library.

Search (`solve`). One function runs either algorithm on the map and returns the path, the number of states expanded, the running time, the number of moves, the total cost and the order in which the treasures were collected. It also records the cell of every expanded state, which the interface uses to draw the exploration.

Map handling. `find_items` locates the start and the treasures; `validate` checks that there is a start, at least one treasure, and that every treasure is reachable, so an impossible map is reported instead of causing a failure; `random_map` builds a bordered map with about 22% random walls and two to four mud patches and is regenerated until every treasure is reachable.

Interface (`Game`). The interface runs at 60 frames per second. After a search it plays two phases: first the explored cells light up as a heat map in the algorithm's colour (cyan for BFS, pink for UCS), then the agent walks the final path, collecting treasures with a small particle effect. The side panel shows the results of both algorithms, marks which is best on moves and on cost, and offers map-editing tools, random and default maps, pause, reset and speed control.



Figure 1: Start screen

6. Experiments and results

6.1 Default map

The default map is 13 x 20 with five treasures.

Algorithm	Moves	Total cost	States expanded	Treasure order (row, col)
BFS	58	78	3394	(9,5) -> (11,7) -> (8,15) -> (6,12) -> (3,9)
UCS	59	59	2609	(9,5) -> (11,7) -> (3,9) -> (6,12) -> (8,15)

BFS finishes in one move fewer, but the extra move UCS makes buys a large saving: BFS's route crosses mud and costs 78, while UCS's route avoids it and costs 59. The two algorithms also choose a different order for the last three treasures.



Figure 2: BFS exploring

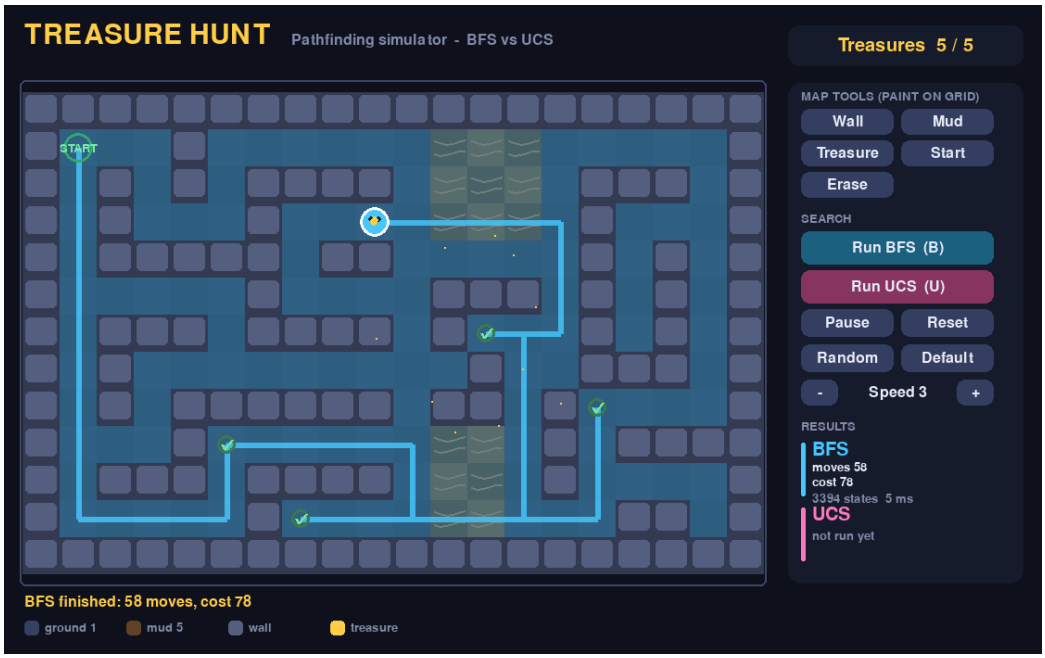


Figure 3: BFS result



Figure 4: BFS vs UCS

6.2 Random maps

To check that the default map is not a special case, we ran both algorithms on random 13 x 20 maps generated by the program. For each number of treasures from 1 to 8 we used 30 maps (240 in total), with the random seed set to 2026. The table shows averages over the 30 maps.

Treasures	BFS states	UCS states	BFS ms	UCS ms	BFS moves	UCS moves	BFS cost	UCS cost	UCS cheaper	BFS fewer moves
1	68	65	0.2	0.3	10.1	10.7	15.0	11.7	12/30	7/30
2	241	234	0.8	1.2	17.9	19.0	31.1	21.8	24/30	11/30
3	574	533	1.9	3.2	23.8	25.8	38.7	29.4	25/30	17/30
4	1311	1238	3.9	5.5	30.1	31.7	43.5	34.5	21/30	15/30
5	2674	2422	10.1	14.4	33.4	35.6	50.1	38.3	25/30	19/30
6	5841	5584	18.5	28.7	37.0	38.9	53.7	45.2	23/30	15/30
7	11468	10913	37.2	59.1	39.1	42.4	59.5	46.5	27/30	17/30
8	26172	23291	83.6	130.2	41.8	44.9	59.5	47.5	27/30	19/30

"UCS cheaper" counts the maps where UCS found a strictly lower total cost than BFS; "BFS fewer moves" counts the maps where BFS used strictly fewer moves. In the remaining maps the two results were equal. Times are from one run on a single computer and depend on the machine.

7. Discussion

- **The two algorithms answer different questions.** In all 240 random maps BFS never used more moves than UCS and UCS never paid a higher cost than BFS. This matches the theory: each algorithm is optimal for its own measure.
- **Terrain cost matters.** For two or more treasures UCS found a cheaper route in 21 to 27 of the 30 maps, and its average cost was about 16 to 30% lower than BFS's. The price was between 0.6 and 3.3 extra moves on average.

- **State growth is exponential.** Because the state includes the set of collected treasures, the number of expanded states grew from about 68 with one treasure to about 26,000 with eight, roughly 2.3 times per added treasure. This is the main limit on the size of problems these algorithms can handle.
- **UCS costs more time per state.** Although UCS expanded slightly fewer states in every row, it took about 1.4 to 1.7 times as long as BFS, because of the priority queue. On these map sizes the average time stayed below 0.14 seconds.
- **A cheaper route can be longer.** The default map shows the practical lesson: the shortest route in moves is not the cheapest in effort when the terrain is not uniform.

8. Limitations and future work

- Movement is four-directional, and the terrain costs are fixed at 1 and 5.
- The search is uninformed. Since the state space doubles with every treasure, maps with many treasures become slow. An informed search such as A* with a good heuristic (for example the distance to the nearest uncollected treasure, or a minimum-spanning-tree estimate over the remaining treasures) would expand far fewer states.
- Possible extensions: diagonal moves, several agents, moving obstacles, a limit on the number of moves, and saving and loading maps.

9. Conclusion

We modelled a multi-treasure pathfinding task as a state-space search and solved it with BFS and UCS. BFS finds the route with the fewest moves and UCS the route with the lowest cost; on the default map the difference is 58 moves at cost 78 versus 59 moves at cost 59. Experiments on 240 random maps agree with this and show the exponential growth in the number of states as treasures are added. The pygame interface makes the behaviour of both algorithms visible and easy to compare, and lets users build their own maps.

10. How to run

```
pip install -r requirements.txt
python3 treasure_hunt_gui.py
```

Click **Run BFS** and **Run UCS** to compare the routes. See the project `README.md` for all controls.

Appendix: reproducing the random-map experiment

Run from the repository folder with `pygame` installed. The script only uses the search and map functions, so no window is opened.

```
import random, statistics as st
import treasure_hunt_gui as g

random.seed(2026)
for k in range(1, 9):
    rows = []
    for _ in range(30):
        m = g.random_map(treasures=k)
        rows.append((g.solve(m, "BFS"), g.solve(m, "UCS")))
    mean = lambda key, i: st.mean(r[i][key] for r in rows)
    print(k, round(mean("expanded", 0)), round(mean("expanded", 1)),
```

```
          round(mean("moves", 0), 1), round(mean("moves", 1), 1),
          round(mean("cost", 0), 1), round(mean("cost", 1), 1))
```

References

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, Pearson (chapter on uninformed search: breadth-first search and uniform-cost search).
2. E. W. Dijkstra, "A note on two problems in connexion with graphs", *Numerische Mathematik*, 1, 269-271, 1959 (the algorithm underlying uniform-cost search).
3. Pygame documentation, https://www.pygame.org/docs/.